

HW1: ER diagram construction, using LLMs :)

In this HW, you are going to do this: create an ER diagram, using 'ollama' to locally (ie. on your laptop) run a pair of LLMs.



Install Ollama from <https://ollama.com/> - it lets you download and run LLMs locally! After it's installed, do this [to install Meta's llama3 model] - **ollama run llama3**:

```
(base) C:\Users\satyc>ollama run llama3
pulling manifest
pulling 6a0746a1ec1a... 1% ▢ 45 MB/4.7 GB 5.0 MB/s 15m17s
```

Similarly, get 'gemma2:2b' [do 'ollama run' or 'ollama pull' on it]. Or, get Gemma 4!

After both models are installed, you are ready to start sending prompts to them :)

pip install these: streamlit, ollama, litellm

Save the following as a .py (eg. invokeMultipleLLMs.py):

```
import streamlit as st
from litellm import completion

# set up the Streamlit app
st.title("Multi-LLM prompting")

# create a text input for user messages
user_input = st.text_input("Prompt:")

if st.button("Send"):
    if user_input:
        messages = [{"role": "user", "content": user_input}]

        # columns for side-by-side display
        col1, col2 = st.columns(2)

        # first
        with col1:
            st.subheader("llama3")
            try:
                response = completion(model="ollama/llama3", messages=messages)
                st.write(response.choices[0].message.content)
            except Exception as e:
                st.error(f"Error: {str(e)}")

        # second
        with col2:
            st.subheader("gemma 2:2b")
            try:
                response = completion(model="ollama/gemma2:2b", messages=messages)
                st.write(response.choices[0].message.content)
            except Exception as e:
                st.error(f"Error: {str(e)}")

        st.divider()

        st.write("Exercise for later – install two more models from https://ollama.com")

        st.divider()
    else:
        st.warning("Please enter a prompt.")

# information about the app
st.sidebar.title("About")

st.sidebar.write("This app, derived from https://github.com/Shubhamsaboo/awesome-llm-apps")

st.sidebar.write(
    "Aim – to create an ER diagram"
)
```



Run the program, by doing this: **'streamlit run invokeMultipleLLMs.py'** - that should pop up your default browser that will display the streamlit app :) Now you can put in prompts that are sent to both LLMs, and look at their outputs side by side - cool!

Multi-LLM prompting

Prompt:

Omg!

Send

llama3

What's up?! It looks like you're excited about something! Would you like to share what's got you so pumped? I'm all ears and ready to chat!

gemma 2:2b

👂 What's going on? 😊

Did something exciting happen, or are you just feeling enthusiastic? 🎉 Let me know! 😊



Put in a prompt [for ER diagram creation - see below for details], and get a screenshot of the LLMs' outputs to the (same) prompt [similar to the screenshot above].

Later you can download and run dozens/100s/1000s/1M+ (!) models from <https://huggingface.co/models> [or <https://modelscope.cn>], and experiment with (small scale) image, audio, video gen too [use the apps in <https://huggingface.co/spaces> as starting points].

Below is a description of the task, of ER diagram creation. 'Normally' you would create the diagram yourself - by coming up with the entities+columns, and relationships between them, and producing a visual output (the ER diagram) that captures DB design. But for this HW, you're using LLMs! So, use what's below to create a (long) prompt for the pair of LLMs. Examine both their outputs, pick what you believe is the better one [you don't need to submit two ER diagrams, just one].

Also, ask for the ER diagram to be output in Mermaid format (a text format that's inspired by Markdown). You'll then copy and paste the Mermaid output from the LLM, into this Mermaid 'renderer' (diagram generator): <https://mermaid.live/edit> - this will produce a visual (diagram), which you'll save to disk, and submit. Make sure the Mermaid syntax looks like so. You can also use <https://www.mermaidflow.app/> [instead of <https://mermaid.live>] if you like, to produce the diagram.

OK, below is the description...

Create a database, to hold data for core operations of a... SMALL COLLEGE!

Sound familiar? Start with <https://bytes.usc.edu/cs585/m26-D-AI-TA/lectures/ER/pics/s41.png> [and <https://bytes.usc.edu/cs585/m26-D-AI-TA/lectures/ER/clips/AllTogetherNow.mp4> to go with it!].

IOW, you need to PROMPT your model (Llama, Gemma...) to make it OUTPUT entities and relations, that mirror the above. In that sense, our HW 'flips' how such HWs are usually assigned: given a description, create an ER diagram. Here, you are given the diagram, and are being asked to describe it to AI so that it can re-output the diagram. BUT that's not all. The given diagram contains only 9 entities, which collectively are about academics - for sure, the core mission of a university. As you know, colleges have MANY other things going on, involving students, professors, non-

faculty. So, augment your description to the AI, to have it account for such additional places/activities/events... (eg. libraries) so that it can output more entities and relationships - to get ideas, LOOK at the map of our campus, SEE/FIND OUT what all is going on (beyond just your classrooms and courses!).

Your descriptions need to make the AI output **6 more** entities (plus relationships) in addition to the 9 above, for a total of **15 entities and their relationships**.

Note that there isn't a single good solution. You are free to make intelligent choices about what data (attributes) to store where (entities), and how to connect all the pieces (relations) - this specifically applies to the 6 'new' entities not shown in the dia. above. ✨

Note too that some data-related requirements you might come up with, might not be able to be captured in an ER diagram - this is fine. Be sure to document your design decisions (they would serve to provide rationale for "why you created your design the way you did").

Please post your qns (and their answers!) on the 'hw1' Piazza page.

What to submit (as a single .zip):

1. full prompt text, plus screenshot [doesn't need to contain the full prompt]
2. full output, plus screenshot [doesn't need to contain the full output]
2. resulting ER diagram (screenshot or saved image, in FULL)

HAVE (LOTS OF AI-ORIENTED) FUN!

Speaking of fun - in the future, do this:

- read some/all of Arthur Hailey's WELL-RESEARCHED, ENGAGING fictional books - about a hospital, airport, hotel, auto industry, pharma...
 - create DB schemas (ie ER diagrams) for each [if YOU ran Ford, what you WOULD want to know about?]
 - have a big LLM ingest AN ENTIRE BOOK, and have -it- output the ER diagram for the underlying operations :)
-
-